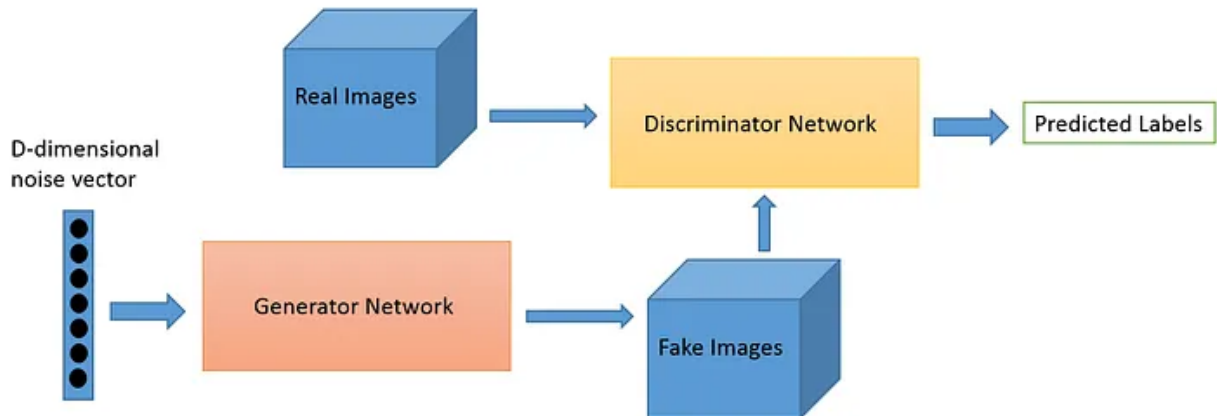


GANs

Han Xiao



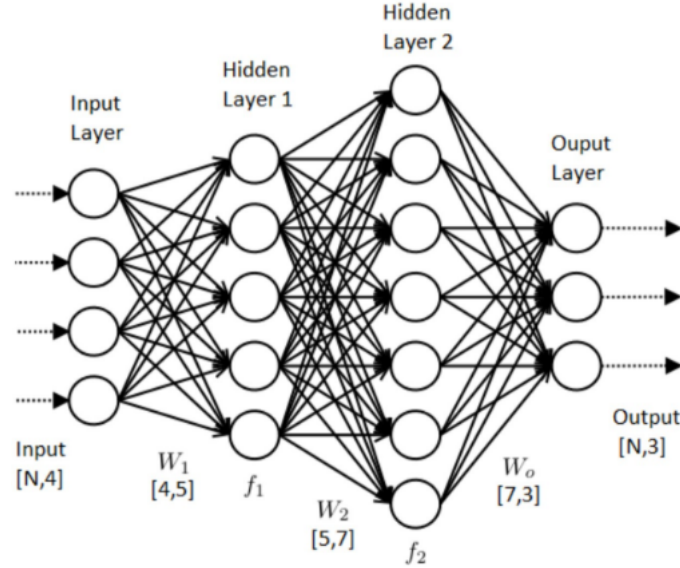
1 Introduction

Generative adversarial networks consist of two parts: a generator network and a discriminator network. The generator creates fake data using random noise chosen from a distribution with the same domain as the preexisting set of real data, while the discriminator acts as a simple binary classifier that tries to determine whether an input is generated or real. The mechanism behind the system is a constant Minmax game between the two networks, where the generator constantly tries to maximize the discriminator's loss (fool the discriminator into thinking a generated image is a part of the real dataset) and minimize its own (generator) loss. This is done through constant weight updates within the neural network. The discriminator utilizes the same method in its own network to counter the generator and output a more accurate probability of the input being real.

2 Neural Networks

The diagram below shows a simplified display of typical networks within the generator and discriminator. The input layer takes the provided information (usually whether or not certain factors are present) and associates the info with a value ranging from 0 to 1 indicating no or yes, respectively. Nodes (circles) from the input layer are connected to the second layer using channels (arrows in the diagram) that each have an assigned weight (W_1), which indicates how important a piece of information is. The following layers of the network are hidden layers, where most of the math required for there to be an output is computed. Notice how some of the nodes are connected by more than one channel. In this case, the value sent to the hidden layer is the sum of the weight times the assigned values of nodes from the input layer. The hidden layer nodes all contain a bias

value, which can be thought of as the constant in a linear equation. Finally, the bias is added to the previously obtained sum and passed through a threshold. If the threshold is met (value $\geq$ threshold), then the node lights up and continues passing input onto the next layer. This process is repeated until the output layer is reached.



In the training process, the weight of each neural network is updated through optimization and regularization methods. Optimization minimizes the cost/loss function of a system by testing various parameter values (for example, weights), while regularization can speed up this process by placing restrictions on the range of values chosen for parameters. In other cases, regularization prevents overfitting: a common problem in training, where the model overgeneralizes based on the given training information, and assumes an incorrect pattern for non-existent data.

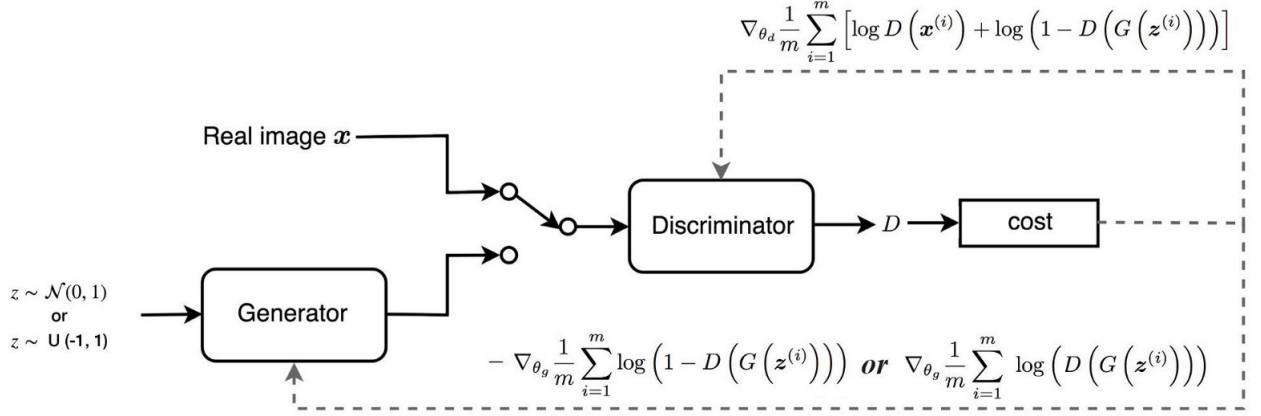
3 The Math Behind GANs

GANs use a minmax loss function that is very similar to the binary cross entropy (*BCE*) function, which penalizes networks for falsely labeling or classifying data. Based on the loss value, weights within the networks are adjusted accordingly. GAN labels data from the preexisting, true dataset with a value of 1 and generated data with a value of 0. It takes values from the generator ($G(z)$) and discriminator ($D(x)$), and multiplies it by the expected value over all instances of a dataset (E). The function $G(z)$ assigns the random noise, z , passed through the generator, a value. Likewise, the function $D(x)$ assigns the data (real or generated) passed through the discriminator, x , a value representing the discriminator's prediction. In *BCE*, Y_i is the ground truth or real value, and $\hat{Y}_i$ is the predicted value. Thus, replacing Y_i and $\hat{Y}_i$ in $Y_i \times \log \hat{Y}_i$ with 1 and $D(x)$, respectively, gives you the first half of the minmax function, or the discriminator's prediction on the real data. To get the second half of the function, or the discriminator's prediction on the fake data, simply replace Y_i and $\hat{Y}_i$ in $(1 - Y_i) \times \log(1 - \hat{Y}_i)$ with 0 and $D(G(z))$ (since the discriminator takes the generator's output as its input for the prediction), respectively. Because the minmax function is taking the whole dataset into account, it must multiply each variable by E , the expected average minmax value of each individual data point. Lower loss values correspond to higher accuracy. Gradient Descent, an optimization method, is then used to minimize the loss function and improve training.

$$L_{BCE} = -\frac{1}{n} \sum (Y_i \times \log \hat{Y}_i + (1 - Y_i) \times \log(1 - \hat{Y}_i))$$

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

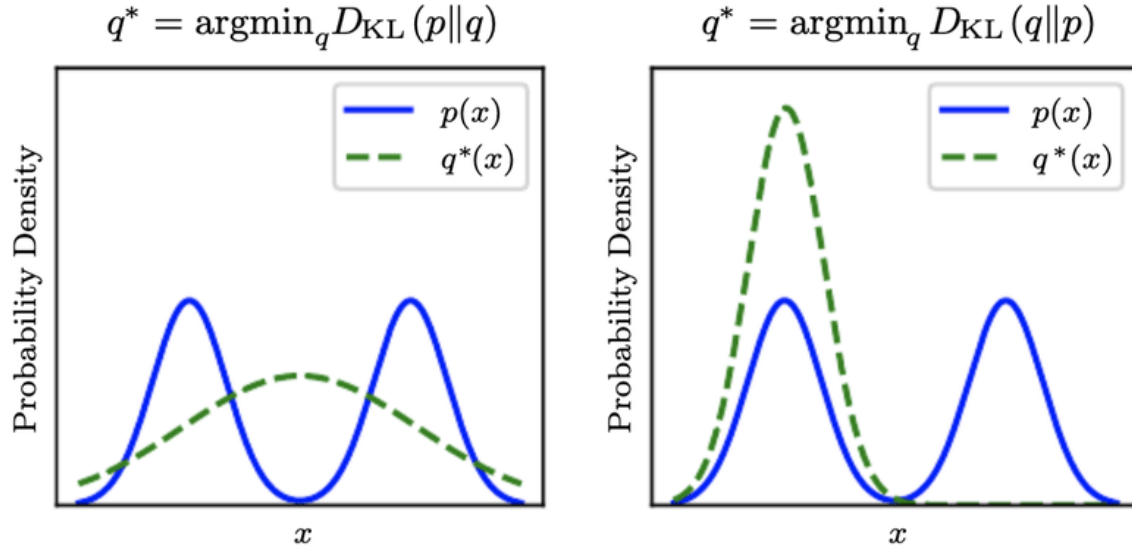
Binary Cross Entropy Function over trials.



Minmax GAN value function.

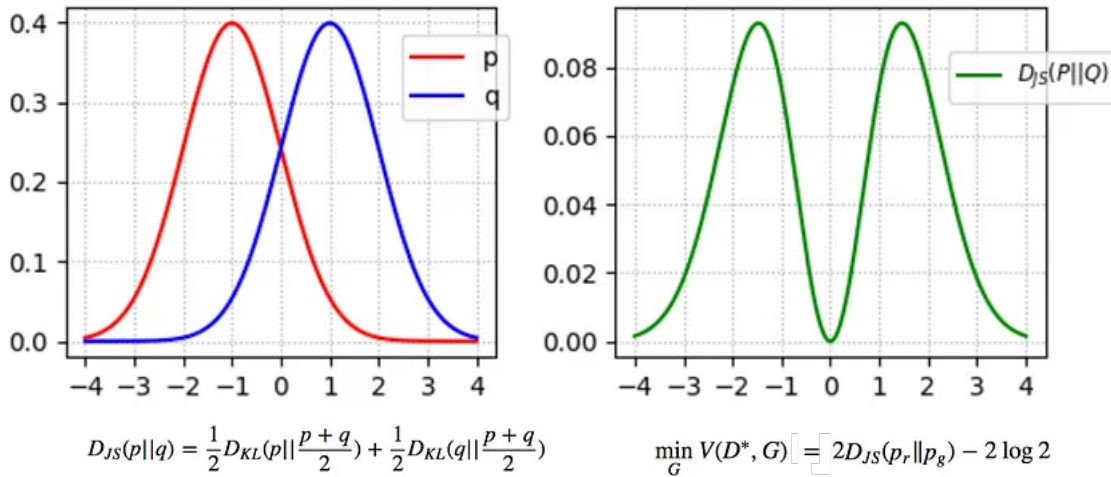
4 Kullback-Leibler Divergence

The Kullback-Leibler divergence measures the difference between two probability distributions of continuous random variables (a variable that has an infinite amount of possible values) using the function $D_{KL}(p||q) = \int_x p(x) \log[p(x)/q(x)] dx$. Simply put, the difference between two distributions can be found by subtracting one from another. The difference can be rewritten in terms of log, where $\log p(x) - \log q(x)$ is the same as $\log[p(x)/q(x)]$. This is then multiplied by a weight, $p(x)$. For continuous random variables, the values can be summed using an integral since each possible value of the variable has a probability of zero, but the regions of values have nonzero probabilities. However, a problem with the KL divergence is that the outcomes differ depending on the distribution used for reference. In other words, $D_{KL}(p||q) \neq D_{KL}(q||p)$. This is especially problematic for datasets that have little to no overlap, or use different parameters, as the cost is extremely low where values of p (actual distribution) are greater than zero and values of q (predicted distribution) approach zero, as shown in the graph below. This produces less diverse samples of higher quality. In the opposite case where p approaches zero and q is greater than zero, the quality is higher, but less diverse samples are produced. At other times, KL divergence approaches infinity. Furthermore, the asymmetrical property of this divergence causes unstable training.



5 Jensen-Shannon Divergence

The Jensen-Shannon Divergence is based on the KL divergence, with an improvement. JS divergence is smoother and has symmetry $D_{KL}(p||q) = D_{KL}(q||p)$, making the training process more stable. In addition, this divergence takes the average of the distributions, and has a range of $[0, 1]$, which solves the problem KL divergence has with approaching infinity when two datasets do not overlap. Interestingly, this is related to Claude Elwood Shannon's Information Theory presented in his article *A Mathematical Theory of Communication*. Information states that unlikely, unexpected, or chaotic events hold more information that is represented using an entropy value, and can be defined as mutual information $I(X; Z) = H(X) - H(X|Z)$, where, in divergence, X is a random variable set to a distribution between P and Q , and Z is a binary indicator variable. The range is a result of constraints due to $H(Z) = 1$ in $\log_2$.



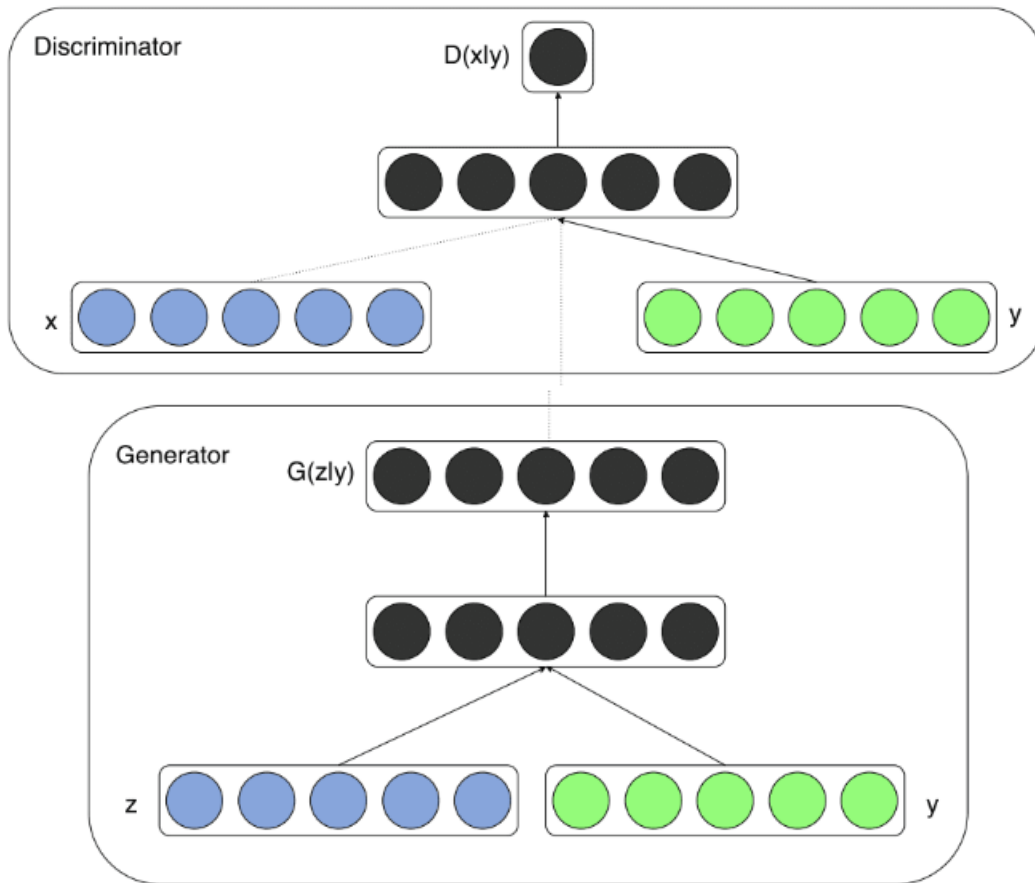
Nonetheless, as long as the data distributions do not overlap at all, the output value will still be the same for two sets of data that are slightly closer to each other, and two sets of data that

have significantly different distributions. To solve this problem, Russian mathematician Leonid Vaseršteĭn's concept of earth mover's distance, or Wasserstein distance, is used. This will be discussed in further detail later on in the article.

6 Types of GANs

6.1 Conditional Generative Adversarial Network (cGAN)

cGANs take additional input such as class labels during the training process, shown by the additional y parameter in the diagram. The additional parameter allows the generator to generate more specific information while assisting in better classification by the discriminator. Conditional Generative Adversarial Networks gave rise to the Pix2Pix model that performs image-to-image translation, which includes tasks such as photo reconstruction and image colorization. This type of network can be useful for synthesizing detailed, high quality photos using text and descriptions, as well as predict upcoming frames in a video based on previous instances.



6.2 Information Maximizing Generative Adversarial Network (InfoGAN)

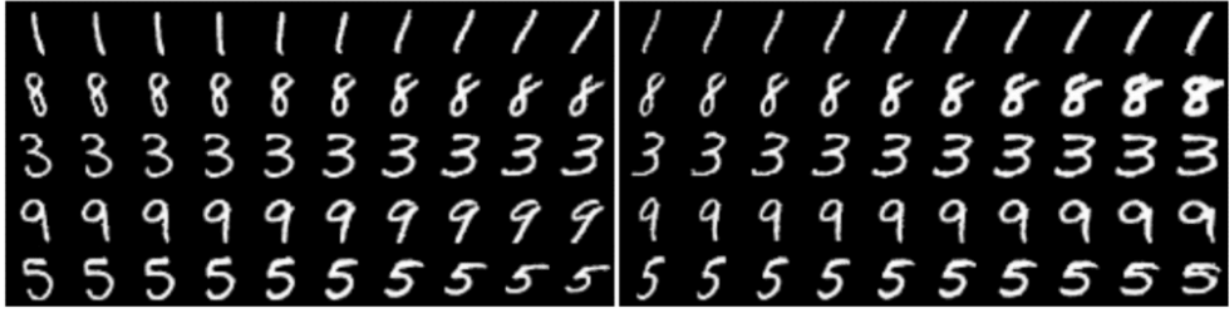
Different from GAN, InfoGAN permits control over the images the generator produces through additional control variables and the use of information theory. Information theory states that the amount of information learned from random variables X and Y is the mutual information between X and Y . To calculate the Mutual Information (MI) loss, the entropy (rate of transfer

of information) of control variable (c) given the generated image using noise (z) and the control variable (c) is subtracted from the entropy of the control variable alone. This mutual information is then used to train the generator in a new model. An example of the usage of InfoGAN can be seen in generated images from the MNIST dataset, where the thickness and rotation of digits within the image is purposely varied.



(a) Varying c_1 on InfoGAN (Digit type)

(b) Varying c_1 on regular GAN (No clear meaning)



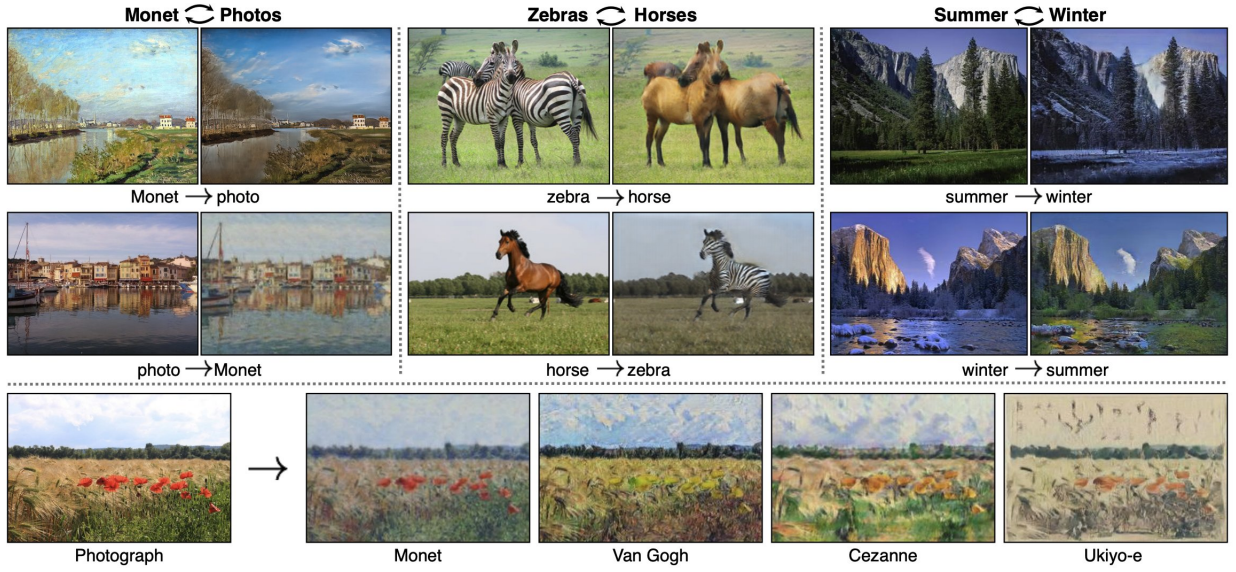
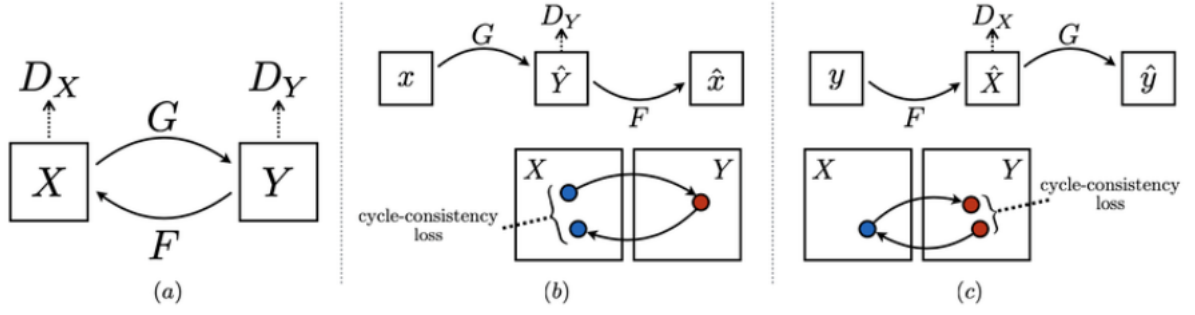
(c) Varying c_2 from -2 to 2 on InfoGAN (Rotation)

(d) Varying c_3 from -2 to 2 on InfoGAN (Width)

Generated images using infoGAN with control variables c_1 (thickness of the digit's stroke) and c_2 (angle of the digit's rotation)

6.3 CycleGAN

CycleGANs are used to perform image to image translation. This is typically done in order to change the style of the image, creating effects like changing seasons, generating paintings from realistic photos, or object transfiguration and applying filters. The math behind this GAN involves the sum of three losses, because two functions are simultaneously being performed. Apart from the usual min-max game between the generator and discriminator, an inverse mapping function is used to map the generated data back onto the original image. There are, in fact, two discriminators and two generators in this model, and they output values based on given domains (ex. winter and spring). One generator uses existing data for domain one to output images for the second domain, while the other generator generates images for the first domain using existing data for the second domain. Abstractly speaking, one generator would be taking pre-existing images of fields in the spring and outputting images of the same fields in the winter, and vice versa. The discriminators then evaluate images and their corresponding datasets.



6.4 Wasserstein Generative Adversarial Network (WGAN)

In WGAN, the discriminator is referred to as the critic, and Wasserstein distance is used instead of K-L or J-S divergence to stabilize training. A Wasserstein distance measures the distance between corresponding data points in separate data sets (the minimum of two data sets, for example), and is calculated using the equation:

$$W(p_{data}, p_G) = \inf_{\gamma} E_{(x,y) \sim \gamma(x,y)} [\|x - y\|]$$

The expected value of the distance between y and x where y and x are distributed according to the optimal choice of joint distribution γ with marginals $\gamma_X(x) = p_{data}(x)$ and $\gamma_Y(y) = p_G(y)$. However, finding a γ that minimizes the distances between p_{data} and p_G is hard to find directly due to complicated distributions and calculations. Instead, the Kantorovich-Rubinstein theorem regarding optimal transport problems is used to produce the following function:

$$W(p_{data}, p_G) = \sup_{\|f\|_{L \leq 1}} E_{p_{data}}[f(x)] - E_{p_G(x)}[f(x)]$$

where a restraint is placed on f by supremum $\|f\|_{L \leq 1}$, and $f(x)$ is similar to that of a lagrange multiplier (a strategy for finding the local maximum and minimum of a function in constrained optimization problems). Essentially, restraint requires the expectation to be calculated over 1

lipschitz ($|f(x) - f(y)| \leq |x - y|$, which places a limit on the slope or speed of the function on how fast one data point can travel to another. This way, a small change in the input only results in a small change in the output. Other forms of lipschitz continuous functions can also be used for bounds that are not equal to 1. The discriminator could be updated using weight clipping, but that requires very specific and exact values. Therefore, it is easier to set a bounded slope similar to before using a lipschitz continuous gradient. The generator is updated to maximize the value of $E_{z \sim p(z)}[f_\theta(G(z))]$.

$$\max_{w \in W} E_{x \sim P_r}[f_w(x)] - E_{z \sim p(z)}[f_w(g_\theta(z))]$$

Below is the original proposed algorithm from the Wasserstein GAN paper:

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α , the learning rate. c , the clipping parameter. m , the batch size. n_{critic} , the number of iterations of the critic per generator iteration.

Require: : w_0 , initial critic parameters. θ_0 , initial generator's parameters.

```

1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w [\frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while

```

7 Disadvantages of The GAN Model

Although GANs can produce surprising results when trained properly, there are many factors and issues these models are prone to during the process. Certain presented problems include: Mode collapse, diminishing gradients, non-convergence, and sensitivity to hyperparameters. Mode collapse occurs when the generator finds a particular weakness in the discriminator. If the discriminator struggles with classifying a distinct type of data, then the generator will only go on to generate data of that type for the rest of the training duration. For instance, if the discriminator struggles with distinguishing real versus fake data for the number 6 when training for number identification, the generator will only produce images of the number 6, leading to a lack of data variation. Diminishing gradients occur when the initial layers of nodes of the network are not updated properly, resulting in less efficient training processes and failure to find proper parameters for deeper layers. Non-convergence is usually a result of the generator being unable to maximize discriminator loss. In other words, the discriminator is doing too well of a job at distinguishing between generated and real data, and thereby unable to tell the generator specific areas in need of improvement. Finally, GANs, like many other networks, are sensitive to hyperparameter tuning.

Examples of hyperparameters include learning rate, number of nodes in a network layer, and the number of network layers. A small change in any of these can have a massive impact on training and data convergence.

8 Conclusion

Although the development of such technology has been particularly useful in healthcare and cybersecurity, its ethics is also highly debated. On one hand, GANs can synthesize medical imaging scans to compensate for the lack of existing data and improve accuracy rates for early cancer detection. This adversarial approach to machine learning can protect systems from attacks and create a safer environment without having to learn from real, preexisting attacks. On the other hand, GANs can also be used to produce realistic drawings, sketches, previously nonexistent song covers, and other art forms at an industrial pace. This presents problems for those in the field, and is inevitably going to lower employment rates.

References

- Tae, Jake. "The Math behind GANs." Jake Tae, March 2020.
<https://jaketae.github.io/study/gan-math/>.
- Brownlee, Jason. "A Tour of Generative Adversarial Network Models." Machine Learning Mastery, July 2019.
<https://machinelearningmastery.com/tour-of-generative-adversarial-network-models/>.
- Normalized Nerd. "The Math behind Generative Adversarial Networks Clearly Explained!" YouTube, 2020.
https://www.youtube.com/watch?v=Gib_kiXgnvA.
- Hui, Jonathan. "GAN — Why It Is so Hard to Train Generative Adversarial Networks!" Medium, Oct. 2019.
<https://jonathan-hui.medium.com/gan-why-it-is-so-hard-to-train-generative-adversary-networks-819a86b3750b>.
- Brownlee, J. How to develop an information maximizing Gan (infogan) in Keras. Machine LearningMastery.com, Jan. 2021.
<https://machinelearningmastery.com/how-to-develop-an-information-maximizing-generative-adversarial-network-infogan-in-keras/#:~:text=The%20InfoGAN%20involves%20the%20addition,via%20mutual%20information%20loss%20function>.
- Godoy, Daniel. "Understanding Binary Cross-Entropy/Log Loss: A Visual Explanation." Towards Data Science, Nov. 2018.
<https://towardsdatascience.com/understanding-binary-cross-entropy-log-loss-a-visual-explanation-a3ac6025181a>.
- Bharath K. "WGAN: Wasserstein Generative Adversarial Networks." Paperspace Blog, Dec. 2021.
<https://blog.paperspace.com/wgans/amp/>.

Arjovsky, Martin, Soumith Chintala, and Léon Bottou. "Wasserstein GAN." Cornell University - arXiv, Dec. 2017.
<https://arxiv.org/abs/1701.07875>.

Das, Shibsankar. "6 GAN Architectures You Really Should Know." Neptune.ai, April 2020.
<https://neptune.ai/blog/6-gan-architectures>.

Brownlee, Jason. "A Gentle Introduction to CycleGAN for Image Translation." Machine Learning Mastery, Aug. 2019.
<https://machinelearningmastery.com/what-is-cyclegan/>.

Alguliyev, Rasim, Ramiz Aliguliyev, and Farhad Yusifov. "Graph Modelling for Tracking the COVID-19 Pandemic Spread." Infectious Disease Modelling, 2020.
<https://doi.org/10.1016/j.idm.2020.12.002>